Leveraging Large Language Models in Cyber Incident Detection and Response Systems

Ing. Juraj Paška
Faculty of Electrical Engineering and Information Technology (FEI)
Slovak University of Technology in Bratislava (STU)
Bratislava, Slovakia
juraj.paska@stuba.sk

***Abstract*—TODO This paper analyzes current trends in the integration of Large Language Models (LLMs) into Security Operations Center (SOC) processes. The study summarizes findings on analyst interactions with GPT-4, examines the effectiveness of multi-model approaches for incident classification, and describes integration possibilities within open-source SIEM platforms. Particular attention is given to Retrieval-Augmented Generation (RAG) techniques and frameworks such as MoRSE, which aim to minimize model hallucinations and increase the accuracy of security threat analysis.**

***Index Terms*—LLM, Cybersecurity, SIEM, SOC, RAG, Incident Detection, SOAR**

I. Introduction

Security Operations Centers (SOCs) serve as the foundation of enterprise cybersecurity defense. The ideal SOC operates through structured tiers of personnel—Tier 1 alert analysts, Tier 2 incident responders, and Tier 3 threat hunters—utilizing an array of security tools including Security Information and Event Management (SIEM), Security Orchestration, Automation, and Response (SOAR), and Endpoint Detection and Response (EDR). However, the real-world execution of these operations is fraught with critical limitations. Current SIEM systems are predominantly rule-based, generating an overwhelming volume of false positives that inevitably lead to severe alert fatigue for Tier 1 analysts. Furthermore, incident response processes remain highly manual, creating operational bottlenecks during critical mitigation phases. These technical challenges are exacerbated by the continuous evolution of attacker tactics, persistent resource shortages, and high staff turnover.

A significant hurdle in modern SOC operations is the sheer volume of unstructured data. Critical threat intelligence, such as Common Vulnerabilities and Exposures (CVEs) or MITRE ATT&CK matrices, exists primarily as unstructured text, making it incredibly difficult for teams to parse and apply manually. Large Language Models (LLMs) present a highly promising solution to address these core limitations. By acting as advanced cognitive assistants, LLMs possess the unique capability to process massive amounts of unstructured data, reduce the cognitive load on human operators, and automate traditionally manual workflows.

The integration of LLMs into cybersecurity operations is currently being explored across four primary research directions:

1) **Incident Detection:** To combat alert fatigue and high false-positive rates, LLMs are utilized to accelerate analysis, prioritize critical alerts, and explicitly explain complex logs by augmenting them with relevant historical and environmental context.
2) **Incident Response:** To resolve the bottlenecks of manual incident handling, LLMs can autonomously generate response actions and guide operators through dynamically adapted remediation playbooks.
3) **Threat Intelligence Analysis:** Instead of relying on manual parsing and report writing, LLMs can automatically analyze unstructured threat intelligence and instantly translate it into actionable SIEM detection rules.
4) **Business Code Analysis:** Moving beyond reactive measures, LLMs are proactively applied to analyze business source code to identify and mitigate underlying vulnerabilities before they can be exploited by threat actors.

By addressing these four domains, LLM-augmented frameworks have the potential to fundamentally transform the SOC from a reactive, overwhelmed monitoring center into a proactive, highly efficient defense mechanism.

II. Analysis of Analyst Interaction with LLMs

Recent research suggests that SOC analysts utilize LLMs primarily as on-demand cognitive aids focused on specific tasks rather than as continuous ”collaborators.” According to a study [1] that analyzed thousands of analyst-generated queries in an operational environment, LLMs are most frequently used for command explanation, scriptwriting, documentation improvement, and data analysis support.

The experiment described in [1] was conducted by capturing all queries sent to the GPT-4 model. A total of over 3,000 queries from 45 analysts were recorded over a 10-month period. A key finding was the growing trend in the frequency of LLM usage in the daily workflow of a SOC analyst, with 11 out of 45 analysts integrating GPT-4 into their everyday routine.

The research showed that only 4% of queries involved a request to directly evaluate an alert as malicious or benign, confirming the use of LLMs as assistive rather than decision-making tools. Furthermore, 75% of interactions consisted of

only 2 to 3 questions, mostly involving iterative text refinement for incident reporting or explanations of functions and logs. The authors highlight the potential for integrating LLM tools directly into SIEM systems to reduce cognitive load by eliminating context switching.

III. Multi-model Approaches and Classification

In [2], the authors explored a multi-model approach to classify security incidents into two categories: interesting and uninteresting. The highest success rate was achieved using GPT-4, reaching a precision of 94% and a recall of 92%. Implementation of this approach led to a 40% reduction in incident processing time and a 60% decrease in cognitive load due to fewer manually handled alerts.

The experimental framework proposed by the researchers consists of a modular architecture featuring an Alert Generation Module, an LLM Agent, and a Reporting Module. This system was integrated into an environment utilizing the Wazuh SIEM platform, where CALDERA was employed for adversary emulation to generate realistic attack scenarios. This setup allowed for a comparative benchmark between five distinct models: GPT-4, GPT-3.5, LLaMA 3, Mixtral 8x22B, and OpenHermes 2.5 Mistral 7B. The inclusion of open-source models like LLaMA and Mixtral was intended to address critical privacy risks and data sovereignty concerns associated with processing sensitive SOC data through external APIs.

Despite the strong performance metrics of the proprietary models, the study highlights a significant operational hurdle: a hallucination rate of approximately 5% during the analysis of security events. While GPT-4 remained the top performer with an F1-score of 93%, the open-source alternatives demonstrated varying degrees of efficacy, trailing behind in complex classification tasks. This performance gap underscores the challenge of balancing model accessibility and privacy with the high accuracy required for autonomous alert triage.

To address these limitations, the authors advocate for a hybrid ”co-pilot” strategy where LLMs act alongside human analysts. They propose further research into prompt optimization, model transparency, and bias detection to improve the reliability of the system. While the current iteration of the framework focuses on weighted outputs and scoring, the authors suggest that future developments should integrate continuous learning from large-scale alert training sets to mitigate the issues of redundancy and correlation between the models in the ensemble.

Despite these benefits, the study has limitations. The reported accuracy figures pertain to individual models, while the success rate of the multi-model voting itself is not clearly specified. Another open point is redundancy; if GPT-4 and GPT-3.5 are highly correlated, the third model’s output may be consistently overruled. Furthermore, the nature of the ”tuning” mentioned is unclear—whether it involved full fine-tuning or merely adjusting weights for final scoring—and a RAG-based approach was not utilized.

IV. Integration into Open-Source SIEM

The authors in [3] present the Security Event Response Copilot (SERC) system, built on the open-source Wazuh SIEM. The system adds two components: a RAG-based module for context-aware incident retrieval and an LLM module for generating recommendations. The RAG system draws information from MITRE ATT&CK, the NIST Cybersecurity Framework (CSF), and an incident response knowledge base.

The architecture of SERC is designed to bridge the gap between strategic risk management and tactical technical execution. By utilizing three distinct vectorized data collections—the NIST CSF 2.0 for high-level lifecycle management, the MITRE ATT&CK framework for granular adversary behavior mapping, and the SOCFortress incident response plan for procedural ground truth—the system provides a multi-layered defense. This integration enables the LLM to deliver recommendations that are not only technically accurate in identifying tactics and techniques but also aligned with established organizational security standards.

The efficacy of the framework was evaluated through an end-to-end simulation using the Atomic Red Team tool to replicate six critical threat categories, including ransomware, malware, and phishing. To assess the quality of the generated responses, the researchers employed multi-faceted metrics such as cosine similarity for semantic alignment and BERTScore for contextual accuracy. This methodology allowed for a rigorous comparison between the proprietary OpenAI models and the open-source Pixtral model, focusing on their ability to minimize hallucinations—a persistent challenge when using LLMs in the SOC space.

The study also compares the performance of Pixtral and OpenAI models. Pixtral demonstrated high stability and consistency, while OpenAI showed greater adaptability but higher performance variability. Specifically, Pixtral consistently achieved higher precision and F1-scores (ranging between 85.4% and 87.2%) across diverse attack scenarios, reflecting its reliability in capturing relevant information without significant fluctuations. In contrast, while the OpenAI model peaked at higher semantic alignment (75.81% cosine similarity in certain cases), it exhibited more pronounced performance dips, suggesting that Pixtral may be a more stable candidate for consistent automated triage in open-source SIEM environments.

V. Advanced RAG Frameworks

Reference [4] addresses the exponential growth of unstructured cybersecurity information and LLM hallucinations. The proposed MoRSE framework is an open-source system integrating dual RAG modules in a cascade: a structured RAG for pre-processed data and an unstructured RAG for raw text. By drawing from CVE repositories, MITRE, and ExploitDB, MoRSE achieved a 15% improvement in general question accuracy and a 50% increase in accuracy for CVE-related incidents compared to GPT-4 without RAG.

The technical distinction of MoRSE (Mixture of RAGs Security Experts) lies in its parallel retriever architecture,

which was inspired by the Mixture of Experts (MoE) paradigm to handle multidimensional cybersecurity contexts. The framework operates in two distinct phases: an Information Retrieval phase, where specialized retrievers collect semantically related data in varied formats, followed by an Answer Generation phase driven by an LLM. Unlike traditional models that rely on static parametric knowledge, MoRSE utilizes a non-parametric knowledge base that allows for real-time updates without the need for resource-intensive retraining. This capability is particularly effective for complex "Multi-Hop" reasoning tasks, where the system must correlate disparate pieces of intelligence to identify root causes or predict subsequent stages of a cyberattack.

VI. Autonomous Agents for Cyber Threat Intelligence

The automation of Cyber Threat Intelligence (CTI) analysis represents a critical frontier in reducing the manual workload of SOC analysts. As explored in [5] and [6], security teams traditionally rely on unstructured reports from sources such as CrowdStrike or MITRE ATT&CK to manually derive correlation rules for SIEM systems. This process is inherently slow and prone to human error. To address this, recent research proposes an autonomous AI agent capable of parsing natural language reports to identify Indicators of Compromise (IOCs) and automatically generating valid regular expressions (RegEx) for direct SIEM integration without human supervision.

The proposed system utilizes a graph-based approach to visualize relationships between IOCs, such as connecting specific file hashes to the domains and IP addresses involved in a singular attack campaign. However, achieving full autonomy requires overcoming four primary challenges: ensuring factual accuracy to filter hallucinations, translating complex natural language into syntactically correct RegEx, performing automated verification of these patterns, and maintaining the structural integrity of IOC relationship graphs.

Experimental results underscore the potential of this approach; in an analysis of over 50 CTI reports, the model identified approximately 2,900 IOCs. After autonomous filtering, 2,300 valid indicators remained, from which 2,200 RegEx patterns were successfully generated. Remarkably, the system missed only 3% of valid IOCs compared to manual ground truth. This shift toward self-learning cybersecurity automation marks a significant milestone in delegating repetitive SOC tasks to autonomous agents, thereby allowing human analysts to focus on higher-level strategic defense.

VII. LLMs in Digital Forensics and Timeline Analysis

Beyond real-time detection and threat intelligence, post-incident investigation—specifically Digital Forensics and Incident Response (DFIR)—presents a significant bottleneck for security teams. Constructing an accurate timeline of a cyber incident is a complex, labor-intensive task that requires parsing thousands of heterogeneous security logs to identify correlations. The authors in [7] address this challenge by introducing GenDFIR, an innovative framework that leverages Retrieval-Augmented Generation (RAG) combined with Large Language Models to automate cyber incident timeline analysis.

By ingesting raw system events (such as Windows Security event logs detailing failed logons, credential validation errors, and audit log clearing) into a dynamic vector knowledge base, GenDFIR enables an LLM to accurately piece together the sequence of adversary actions. This methodology overcomes the traditional limitations of manual log triage, significantly accelerating the reconstruction of attack paths while mitigating human error and analyst fatigue. The RAG architecture is crucial in this context, as it anchors the LLM's outputs strictly to the ingested forensic artifacts, thereby preventing hallucinations and maintaining the evidentiary integrity of the analysis.

Evaluations of the GenDFIR framework demonstrated its capability to successfully reconstruct complex scenarios, such as unauthorized access chains, by maintaining strict chronological coherence and contextual relevance. By automating the extraction and chronological mapping of crucial artifacts, the system produces high-fidelity timeline reports that are directly applicable for formal incident reporting, root-cause analysis, and the formulation of subsequent defensive strategies.

VIII. Predictive Threat Intelligence Models

Reference [8] investigates the integration of LLMs to overcome the limitations of static, signature-based detection systems that frequently struggle with zero-day exploits and polymorphic threats. The authors introduce the Predictive Threat Intelligence Model (PTIM), a framework trained on approximately 100 million security-related data points, including network logs and public threat databases. By utilizing the bidirectional context awareness inherent in transformer architectures, PTIM can recognize linguistic and non-linguistic patterns indicative of malicious intent that traditional models often overlook.

Empirical validation shows that PTIM achieves a detection accuracy exceeding 95% with a substantial reduction in false positives compared to rule-based systems. A key technical advantage of this framework is its operational efficiency; it performs significantly faster than standard LLM architectures, allowing for the near-instantaneous identification of ransomware attacks. This capability enables organizations to transition from a reactive posture to a proactive defense strategy, identifying and mitigating threats before they can cause widespread system compromise.

IX. LLM-Based Agents for Tier 1 Triage Automation

While previous research emphasizes broad intelligence gathering, [9] provides a specific evaluation of LLM agents within the critical triage process of Tier 1 SOC analysts. The study proposes a structured framework where an LLM agent serves as an intermediary between a SIEM platform (Wazuh) and the human analyst. By testing five prominent models—including

OpenAI's GPT-4o, Meta's Llama 3, and Mixtral 8x22B—the research investigates the agents' ability to autonomously classify alerts as either "interesting" or "not-interesting" based on raw log data. This classification logic is intended to filter out the high volume of "noise" generated by normal user operations, allowing analysts to focus on high-fidelity threats identified through adversary emulation tools like CALDERA.

Experimental findings indicate that LLM agents significantly enhance the speed of initial triage and reduce the cumulative cognitive load on security staff by providing natural language explanations for each alert classification. Despite these gains, the research highlights critical limitations such as the propensity for occasional hallucinations and the inherent privacy risks of processing sensitive log data through external models. Consequently, the study concludes that while LLM agents are highly effective for routine automation, they are best deployed in a "co-pilot" capacity. This hybrid approach ensures that human expertise remains in the loop to verify LLM reasoning, thereby maximizing operational efficiency while maintaining the rigorous security standards required in a modern SOC environment.

X. Optimization of SIEM Detection Rule Sets

While the automated generation of rules significantly expands detection coverage, it often introduces a secondary challenge: the proliferation of redundant or overlapping rules. Such overlaps increase computational overhead, response latency, and, most critically, alert fatigue for SOC analysts. Reference [10] introduces RuleGenie, a novel LLM-aided recommender system designed to optimize enterprise-level SIEM rule sets. The authors argue that manual optimization is increasingly unsustainable due to the sheer scale of modern rule bases, which can contain thousands of platform-specific queries (e.g., KQL, Sigma, or SPL) with complex logic.

The RuleGenie framework leverages the multi-head attention capabilities of transformer models to generate high-dimensional embeddings of SIEM rules. By applying a similarity matching algorithm to these embeddings, the system identifies the top-k most similar rules within a repository. Furthermore, RuleGenie utilizes Chain-of-Thought (CoT) reasoning to analyze the underlying logic of flagged rules, allowing it to recommend consolidations or deletions of redundant logic. This optimization process ensures that the SIEM maintains a lean and efficient detection posture, reducing the signal-to-noise ratio and ensuring that the most relevant alerts are prioritized for human investigation.

XI. Automated Mapping of SIEM Rules to Adversarial Techniques

While the generation and optimization of SIEM rules are vital for visibility, the effectiveness of a security posture is often measured by its alignment with standardized frameworks like MITRE ATT&CK. As explored in [11], manual mapping of structured detection rules—such as those written in Splunk's Search Processing Language (SPL)—to specific Tactics, Techniques, and Procedures (TTPs) is a labor-intensive process prone to subjective error. To resolve this, the authors propose Rule-ATT&CK Mapper (RAM), a multi-stage pipeline that utilizes Large Language Models to autonomously categorize structured rules. RAM is unique in its "prompt chaining" approach, which first translates technical rule syntax into natural language descriptions enriched by external contextual information (e.g., IoC significance) before performing the final technique recommendation.

Experimental evaluations using the Splunk Security Content dataset demonstrate that RAM significantly improves mapping accuracy without the need for model fine-tuning. By testing multiple models, including GPT-4-Turbo and IBM Granite, the study found that incorporating a web-search agent to retrieve supplementary contextual knowledge about Indicators of Compromise (IoCs) is essential for overcoming the limitations of an LLM's implicit domain knowledge. The framework achieved an F1-score of 0.62 and a recall of 0.75, highlighting its capability to assist SOC teams in maintaining an up-to-date and accurate coverage map of the ATT&CK matrix. This automation allows security engineers to rapidly validate whether their detection rule base provides sufficient protection against specific adversarial strategies.

XII. Multi-Agent Modular Architectures for Correlated Threat Detection

As cyberattacks become increasingly multi-vector and stealthy, isolated detection systems often fail to correlate related events across different layers of an organization's infrastructure. To address this, reference [12] proposes a modular multi-agent architecture that integrates domain-specific cybersecurity tools with the semantic reasoning capabilities of Large Language Models. The framework consists of three specialized autonomous agents—focused on email verification, server log analysis, and IP range scanning—which independently process data using tailored detection pipelines. These agents translate raw technical outputs into structured risk signals, which are then fed into a centralized contextual recommendation system.

The core innovation of this architecture lies in its ability to synthesize disparate proofs from individual agents into a coherent attack narrative. By cross-analyzing signals (e.g., correlating a suspicious phishing email with subsequent lateral movement in server logs), the system can detect complex threat patterns that evade standalone sensors. Experimental results on benchmark datasets, including CIC-IDS 2017 and SpamAssassin, demonstrate a high detection accuracy of 93.6% and a 41.3% reduction in false positives compared to traditional rule-based methods. Furthermore, the use of LLM-based Chain-of-Thought (CoT) reporting provides explicable insights for security analysts, significantly reducing triage time and increasing confidence in the system's automated recommendations.

XIII. Our work - Multiagent incident response automation

To address the inherent limitations of modern Security Operations Centers (SOC), we propose the Multiagent Incident Response Assistant (MIRA). MIRA is designed as an

autonomous framework to assist analysts during the incident reaction phase by leveraging specialized Large Language Model (LLM) agents and advanced retrieval techniques.

A. System Architecture and Tech Stack

The MIRA framework is built upon a modular infrastructure deployed on the Pyctus environment. The core orchestration is handled via *LangGraph*, which enables a multiagent architecture, while *Chainlit* provides the user interface for human-operator interaction. The backend is powered by *Django* and *PostgreSQL*, the latter specifically utilized for storing embeddings and performing vector operations such as cosine similarity.

The system integrates a playbook database consisting of approximately 90 playbooks across various security categories, managed via *MediaWiki* and *Cacao Roaster*. For local model execution and management, *Ollama* is employed, ensuring that sensitive security data can be processed on-premises or within restricted hardware environments.

B. Hybrid Search and RAG Methodology

The core of the MIRA framework is a Retrieval-Augmented Generation (RAG) pipeline that leverages a hybrid search approach to maximize the relevance of retrieved security playbooks. This system distinguishes between two primary search vectors:

- **Full-text Search (FT):** This branch focuses on explicit technical identifiers, such as specific CVE IDs, attack categories, or unique security keywords. We implement the **BM25 (Best Matching 25)** algorithm, which ranks documents based on the appearance frequency of search terms relative to the entire playbook repository.
- **Semantic Search (SEM):** This branch enables retrieval based on the underlying intent and context of the query. Security playbooks are vectorized in their entirety with the `all-MiniLM-L6-v2` embedding model and stored in an *in-memory* vector database. Queries are also vectorized using the same model to ensure consistent similarity scoring. To calculate the relationship between the query vector and playbook embeddings, we utilize **cosine similarity**.

C. Result Fusion and Configuration

The retrieval system is highly configurable, with the primary parameter K defining the total number of returned records. To integrate the results from the FT and SEM branches, we implement three distinct fusion methodologies:

1) **Naive Approach:** A simple division of the parameter K in a fixed ratio (e.g., $K_{FT} = 2, K_{SEM} = 3$). This serves as a baseline to test the impact of each method without complex mathematical merging.
2) **Weighted Average:** Results from both branches are first normalized into the interval $[0, 1]$. The final relevance score is calculated as:

$$Score = (W_{SEM} \cdot Norm_{SEM}) + ((1 - W_{SEM}) \cdot Norm_{FT}) \quad (1)$$

where W_{SEM} is a configurable weight (defaulting to 0.5) that balances the influence of semantic context against keyword precision.

3) **Reciprocal Rank Fusion (RRF):** As illustrated in our framework's optimization phase, we utilize RRF to combine the rankings from multiple search algorithms without requiring score normalization. The fused score for a document d within the set of all documents D is calculated as:

$$RRFscore(d \in D) = \sum_{r \in R} \frac{1}{k + r(d)} \quad (2)$$

where R is the set of rankings (full-text and semantic), $r(d)$ is the rank of document d in ranking r, and k is a constant (typically set to 60) used to mitigate the impact of low-ranked outliers.

D. Testing of search algorithms and Evaluation Framework

To validate the effectiveness of our retrieval and fusion logic, we developed a testing suite consisting of five reference queries with predefined "ground truth" playbooks. We evaluate the system's performance using a multi-metric scoring system:

- **Order-based Score:** A scoring system where a retrieved target playbook is assigned points based on its position ($5 - IDX$ for the TOP 5 results).
- **Hit Rate:** A binary metric indicating whether the desired playbook is present within the top K results.
- **Precision@K:** The proportion of relevant playbooks found within the top K retrieved items.
- **Mean Reciprocal Rank (MRR):** The average of the reciprocal ranks of the first relevant playbook, prioritizing configurations that place the correct solution at the top of the list.

By experimentally comparing the performance of the semantic and full-text branches independently against the hybrid configurations, we demonstrate the necessity of the hybrid approach in capturing threats that lack explicit keyword signatures or clear semantic definitions.

E. Testing Models – LLM as Judge

To identify the most effective models for our multi-agent architecture, we developed a specialized testing framework designed to quantitatively evaluate LLM performance within the cybersecurity domain. The framework is built with a focus on modularity, allowing for the seamless swapping of models via *Ollama*, the adjustment of system prompts, and the toggling of RAG capabilities.

1) Testbed Architecture and Configuration: The evaluation system utilizes *Dependency Injection* to integrate the hybrid search module defined in previous sections. This allows the testing script to call a standardized interface:

```
public interface SearchInterface {
    public Collection<SearchResult> Handle(QueryTy
}
```

By decoupling the search logic from the model evaluation, we can strictly measure how different models utilize the retrieved context. For this study, we evaluated at least three different models in two primary states: (1) **Plain LLM (Zero-shot)**, where the model relies solely on its internal training data, and (2) **LLM + RAG**, where the model's prompt is extended with relevant playbook segments retrieved from our database.

2) Evaluation Metrics and Agent Roles: We define specific success criteria based on the three functional sub-agents within the MIRA architecture. To determine the most effective configuration, each underlying model was tested in two states: as a standalone LLM and as an LLM augmented with RAG. The evaluation focuses on the following roles:

- **Orchestrator Evaluation:** We measure the model's ability to categorize an incident correctly and map the analyst's query to the appropriate sub-agent (Todo, Action, or Chat). The primary metric is classification accuracy.
- **Sub-agent Evaluation:**
  - **Todo Agent:** Evaluated on its ability to break down complex security incidents into logical, sequential steps based on playbook requirements.
  - **Action Agent:** Assessed on its ability to generate concrete technical commands and generalize entities—for example, correctly identifying and replacing a placeholder IP address from a template with the specific IP found in the incident log.
  - **Chat Agent:** Evaluated on its ability to provide concise, context-aware explanations and maintain a coherent dialogue with the SOC analyst.

Unlike traditional benchmarks, we do not utilize automated mathematical metrics such as BERTscore or Cosine Similarity, as these often fail to capture the functional correctness required in a security context. Instead, all qualitative outputs are quantified exclusively through the LLM-as-Judge paradigm.

3) LLM as Judge: A cornerstone of our evaluation methodology is the **LLM as Judge** paradigm, utilizing the **Ministral-3 (14B)** model as the autonomous arbiter. Recognizing that metrics like ROUGE or BLEU are insufficient for technical security instructions, the Judge model evaluates the outputs of our specialized workers based on technical accuracy, adherence to the retrieved playbook, and logical consistency.

During the testing phase, we benchmarked several high-performance models for each role:

- **Todo Agent:** Evaluated using **Qwen3 (14B)** with a 40K context window.
- **Action Agent:** Evaluated using **Phi-4 (3.8B)** with a 128K context window.
- **Chat Agent:** Evaluated using **Llama 3.1 (8B)** with a 128K context window.

This approach allowed for a scalable comparison of how each model's reasoning capabilities improved when provided with RAG-retrieved context versus their baseline zero-shot performance.

F. Multiagent Orchestration and Task Splitting

MIRA utilizes a "Supervisor" architecture to manage task delegation. In this paradigm, a central orchestrator receives the analyst's query, identifies the intent, and manages the handoff to the specialized worker agents. We define our primary sub-agents as follows:

- **Todo Agent:** Responsible for workflow management and the generation of structured task lists (To-Dos) for incident resolution.
- **Action Agent:** Acts as the technical generator, drafting specific response instructions and commands based on identified playbooks.
- **Chat Agent:** Serves as the conversational interface, allowing analysts to query the system for context, summaries, or clarification on specific alerts.

This architecture (1) maintains visual and logical separation between agents, allowing for specialized context management. The supervisor ensures that agent outputs are validated before being presented to the operator, significantly reducing the cognitive load while maintaining human-in-the-loop oversight.

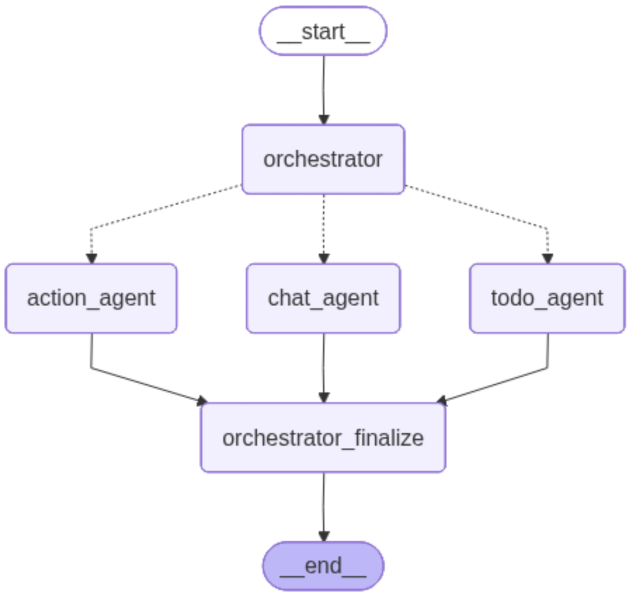

Fig. 1. Multiagent orchestration and task splitting in MIRA.

G. Multiagent Orchestration and Task Splitting

MIRA utilizes a "Supervisor" architecture to manage task delegation. In this paradigm, a central orchestrator (Supervisor) receives the analyst's query, maps it to the correct worker, and manages the handoff. We define specialized worker roles including:

- **Chat Agent:** Provides domain-specific knowledge and categorization of incidents.
- **Todo Agent:** Drafts specific response instructions based on identified playbooks.
- **Action Agent:** Maps instructions to concrete SOC actions and tools.

This architecture maintains visual and logical separation between agents, allowing for specialized context management using techniques like sliding windows or history summaries. The supervisor ensures that agent outputs are validated against the ground truth before being presented to the operator, significantly reducing the cognitive load while maintaining human-in-the-loop oversight.

H. Infrastructure and CI/CD

The development lifecycle is supported by a robust CI/CD pipeline integrated with GitLab. Every merged Pull Request triggers an automated build of a Docker image for the Python application, which is then deployed to the Pyctus infrastructure via SSH. This ensures that the assistant remains up-to-date with the latest research-driven optimizations in search algorithms and model configurations.

XIV. Experimental Results and Analysis

To determine the optimal configuration for the MIRA framework, we conducted a series of benchmarks comparing three Large Language Models—**Qwen3 (14B)**, **Llama 3.1 (8B)**, and **Phi-4 (3.8B)**—across our three specialized agent roles. The evaluation was split into baseline performance (without RAG) and augmented performance (with RAG). The results, summarized in Table I, highlight the varying impact of retrieval-augmented context on different task types.

TABLE I
Average performance scores by Agent and Model (Scale 0.0–1.0)

Agent	Model	Without RAG	With RAG
Chat Agent	Qwen3 (14B)	**0.86**	0.56
	Llama 3.1 (8B)	0.61	0.45
	Phi-4 (3.8B)	0.58	0.63
Action Agent	Llama 3.1 (8B)	0.00	**0.31**
	Phi-4 (3.8B)	0.02	0.25
	Qwen3 (14B)	0.00	0.17
Todo Agent	Qwen3 (14B)	0.25	**0.32**
	Phi-4 (3.8B)	0.24	0.26
	Llama 3.1 (8B)	0.23	0.17

A. Analysis of Findings

The experimental data reveals several critical insights regarding the behavior of LLMs in a SOC environment:

1) The Necessity of RAG for Technical Tasks: For the **Action Agent**, performance without RAG was negligible, with scores ranging from 0.00 to 0.02. Without access to the specific syntax and parameters defined in the security playbooks, the models consistently failed to produce valid machine-readable JSON or correct technical commands. Upon enabling RAG, the **Llama 3.1 (8B)** model showed the most significant improvement, reaching a score of 0.31. This confirms that RAG is a mandatory requirement for agents tasked with technical command generation.

2) The Verbosity Penalty in Conversational Tasks: A surprising trend was observed in the **Chat Agent** role, particularly with the **Qwen3** and **Llama 3.1** models. Both models saw a decrease in performance when RAG was enabled. The reasoning provided by the *Ministral-3* judge indicated a "verbosity penalty"; the inclusion of retrieved playbook context often led the models to generate responses that were overly detailed or included unnecessary procedural information, drifting away from the directness required in a conversational setting. Only the **Phi-4 (3.8B)** model managed to improve its chat score with RAG (0.58 to 0.63).

3) Model Selection for Production: Based on these results, we selected a heterogeneous model strategy to optimize the MIRA framework. **Qwen3 (14B)** was selected for the **Todo Agent** due to its superior planning score with RAG (0.32). **Llama 3.1 (8B)** was identified as the best candidate for the **Action Agent** based on its technical extraction capabilities. For the **Chat Agent**, although **Qwen3** had the highest standalone score, **Llama 3.1** was selected to maintain a balance between context-awareness and resource efficiency, leveraging its 128K context window for long-term dialogue stability.

B. Qualitative Observations

The LLM-as-Judge noted that while RAG significantly improved grounding, models still encountered difficulties with *temporal relevance*. In high-pressure scenarios (e.g., immediate response to a ransomware note), models sometimes prioritized long-term recovery steps over immediate containment actions. This suggests that further refinement of the system prompts is necessary to ensure that the priority of tasks matches the urgency of the security incident.

XV. Limitations and Future Work

Limitations - only LLM as judge. Better model could be used for evaluation. Also cosine similarity and feedback from human analysts could be used to further validate the results. - playbooks are vectorized as a whole, not at the level of individual steps. This can lead to irrelevant information being included in the prompt, which may cause hallucinations or confusion for the LLM. - evaluation questions were in most cases out of scope of the playbooks, which may have led to lower performance in the zero-shot configuration. This also highlights the importance of RAG in providing relevant context to the LLM. - sliding window with appended history is used to manage context, which may not be optimal for all types of queries or incidents. More sophisticated context management techniques could be explored in future work - prompt engineering is a critical factor in the performance of the LLM agents, and further optimization of prompts could lead to improved results. This includes experimenting with different prompt structures, instructions, and examples to better guide the LLM's reasoning process. - training was not considered, also feedback is not currently used to improve the model's performance over time. Implementing a feedback loop where the system learns from past interactions and outcomes could enhance its accuracy and relevance in future responses.

- orchestrator (supervisor) now just delegates tasks based on intent classification, but it could be enhanced to perform more complex reasoning (reasoning loops, validations etc..)

XVI. Conclusion

TODO The integration of LLMs into SOC processes represents a significant shift in the efficiency of cyber threat analysis. Future research should focus on deeper RAG integration and the reduction of analyst cognitive load by minimizing context switching between LLM interfaces and primary security tools.

References

[1] X. Zhang, Y. Li, and H. Chen, "An empirical study on soc analyst interaction with large language models," arXiv preprint arXiv:2508.18947, 2025, online. Available: https://arxiv.org/abs/2508.18947.

[2] R. Singh, A. Patel, and S. Kumar, "A multimodel approach for enhancing siem performance using llms," Research Square, 2025, online. Available: https://www.researchsquare.com/article/rs-5615639/v1.

[3] L. García and M. Torres, "Integrating ai agents into open-source siem platforms: A wazuh-based framework," *Sensors*, vol. 25, no. 3, p. 870, 2025, online. Available: https://www.mdpi.com/1424-8220/25/3/870.

[4] D. Kim, L. Zhao, and J. Park, "Morse: Multi-dimensional cybersecurity reasoning via dual rag frameworks," arXiv preprint arXiv:2407.15748, 2024, online. Available: https://arxiv.org/pdf/2407.15748v1.

[5] A. Surname *et al.*, "Autonomous cyber threat intelligence reporting and ioc extraction via large language models," arXiv preprint arXiv:2407.13093, 2024.

[6] Anonymous, "Poster: Towards autonomous ai agents for soc analyst tasks," Proc. IEEE Symposium on Security and Privacy (S&P), 2025, online. Available: https://sp2025.ieee-security.org/downloads/posters/sp25posters-final24.pdf.

[7] F. Y. Loumachi, M. C. Ghanem, and M. A. Ferrag, "Advancing cyber incident timeline analysis through retrieval-augmented generation and large language models," *Computers*, vol. 14, no. 2, p. 67, 2025, online. Available: https://www.mdpi.com/2073-431X/14/2/67.

[8] B. Bokkena, "Enhancing it security with llm-powered predictive threat intelligence," in *Proc. 2024 5th International Conference on Smart Electronics and Communication (ICOSEC)*, 2024, pp. 751–756.

[9] O. Oniagbi, "Evaluation of llm agents for the soc tier 1 analyst triage process," Master's thesis, Dept. of Computing, Univ. of Turku, Turku, Finland, 2024, available: https://www.utupub.fi/handle/10024/177651.

[10] A. Shukla, P. A. Gandhi, Y. Elovici, and A. Shabtai, "Rulegenie: Siem detection rule set optimization," arXiv preprint arXiv:2505.06701, 2025.

[11] S. Wudali, A. Shukla, P. A. Gandhi, Y. Elovici, and A. Shabtai, "Rule-att&ck mapper (ram): Mapping siem rules to ttps using llms," arXiv preprint arXiv:2502.02337, 2025.

[12] Y. Hmimou, M. Tabaa, A. Khiat, and Z. Hidila, "A multi-agent system for cybersecurity threat detection and correlation using large language models," *IEEE Access*, vol. 13, pp. 31 751–31 766, 2025.